

UNIVERSIDAD AUTÓNOMA DE YUCATÁN

Facultad de Matemáticas – Campus de Ciencias Exactas e Ingenierías
Mérida, Yucatán, México

Asignatura: Inteligencia Artificial

Profesor: Dr. Víctor Uc Cetina

Ejercicio 1: Modificación del Entorno Wumpus World

Fecha de entrega: 27 de agosto de 2026

1. Configuración del Mapa (config/mi_cueva_4x4.yaml)

A continuación se presenta el código de la cueva diseñada, en el cual se modificaron las posiciones originales del Wumpus, los pozos y el oro, garantizando un camino seguro de ida y vuelta a la casilla inicial (1,1).

```
1 grid:
2   width: 4
3   height: 4
4
5 agent:
6   start: [1, 1]
7   direction: east
8   arrows: 1
9
10 wumpus: [4, 1]
11
12 pits:
13   - [2, 3]
14   - [4, 3]
15   - [3, 4]
16
17 gold: [1, 3]
18
19 scoring:
20   step: -1
21   gold: 1000
22   death: -1000
23   shoot: -10
24
25 max_steps: 100
```

Listing 1: Archivo config/mi_cueva_4x4.yaml

2. Diagrama del Entorno (Cueva 4x4)

Representación matricial del mundo de Wumpus diseñado (coordenadas 1-based, con origen (1,1) en la esquina inferior izquierda).

	1	2	3	4	(Columna X)
4	.	.	P	.	
3	G	P	.	P	
2	.	.	.	.	
1	>	.	.	W	
(Fila Y)					

Leyenda:

> : Agente en posición inicial (1,1) mirando al Este
 W : Wumpus en (4,1)
 P : Pozos (Pits) en (2,3), (4,3) y (3,4)
 G : Oro (Gold) en (1,3)
 . : Casilla vacía

Ruta segura identificada para obtener el oro y salir:

(1,1) -> (2,1) -> (3,1) -> (3,2) -> (2,2) -> (1,2) -> (1,3) [Grab] ->
 (1,2) -> (1,1) [Climb]

3. Reporte de Resultados y Análisis

3.1. Resultados por Agente

- **02_simple_reflex_agent: FALLA** (Muere en pozo o no logra salir). No posee memoria ni mapa interno. Al percibir brisa gira indefinidamente.
- **03_model_based_agent: ÉXITO** (+982 pts aprox). Mantiene mapa de seguridad inferido y explora solo casillas 100% seguras.
- **04_goal_based_agent: ÉXITO** (+984 pts aprox). Utiliza búsqueda BFS sobre casillas seguras para recuperar el oro y volver al inicio.
- **05_utility_based_agent: ÉXITO** (+984 pts aprox). Maximiza el valor esperado planificando la ruta de mayor puntaje.
- **06_learning_agent: ÉXITO** (Tras 1500 épocas). Aprende la ruta óptima mediante Q-Learning.

3.2. Preguntas de Análisis

1. ¿Qué agentes lograron salir con el oro en tu mapa y cuáles no? Los agentes ****basados en modelo (03)****, ****basados en metas (04)****, ****basados en utilidad (05)**** y el ****agente de aprendizaje (06)**** lograron obtener el oro y escapar de la cueva con puntajes positivos. Por el contrario, el ****agente de reflejo simple (02)**** falló en cumplir la misión.

2. ¿Por qué el agente de reflejo simple falla (o tiene suerte) en tu diseño? El agente de reflejo simple no mantiene un modelo o memoria de la cueva ni registra las casillas visitadas. Su política de acciones responde únicamente a las percepciones inmediatas: ante un brisa (*breeze*) o hedor (*stench*), reacciona girando sobre su propio eje. Al llegar a la casilla (2, 2) y percibir la brisa emitida por el pozo en (2, 3), empieza a girar y termina avanzando a ciegas hacia una casilla fatal o repitiendo ciclos sin ser capaz de recordar dónde se ubica la salida (1, 1) para ejecutar la acción *Climb*.

3. ¿Cómo cambia el resultado del agente basado en modelo si acercas o alejas un pit de la casilla inicial? Si se coloca un pozo en la casilla adyacente (1, 2) o (2, 1), el agente basado en modelo percibe brisa inmediatamente en el estado inicial (1, 1). Dado que el agente sigue una política estrictamente cautelosa y no puede deducir con certeza cuál de las dos casillas contiguas contiene el pozo sin arriesgarse, se rehúsa a avanzar y se queda girando infinitamente en la casilla (1, 1). Al alejar los pozos del inicio, el agente dispone de suficiente información de adyacencia segura para explorar y resolver la cueva.

4. Evidencias de Ejecución

A continuación se adjuntan las capturas de pantalla que demuestran la ejecución del entorno y los agentes sobre la configuración `config/mi_cueva_4x4.yaml`.

```

91  ```bash
92  python 02_simple_reflex_agent.py --config
    config/mi_cueva_4x4.yaml
93  python 03_model_based_agent.py --config config/
    mi_cueva_4x4.yaml
94  python 04_goal_based_agent.py --config config/
    mi_cueva_4x4.yaml
95  python 05_utility_based_agent.py --config
    config/mi_cueva_4x4.yaml
96  ```
97
98  ## Criterios de aceptación
99
11  arrows: 1
12
13  wumpus: [1, 3]
14
15  pits:
16    - [3, 1]
17    - [3, 3]
18    - [4, 4]
19
20  gold: [2, 3]
21
22  scoring:
23    gold: 1000

Step 100  Score -100.0  IN CAVE
 4 | . . P .
 3 | G P . P
 2 | . . . .
 1 | . . < W
    1 2 3 4
Percept [Stench]
Reward -1 (max_steps)
-----
Result: stopped without gold  steps=100  score=-100.0
PS C:\Users\marle\Git_IA\inteligencia-artificial\Agentes\project>
  
```

Figura 1: Ejecución del visor del entorno (01_wumpus_world.py).

```

93  python 03_model_based_agent.py --config config/
    mi_cueva_4x4.yaml
94  python 04_goal_based_agent.py --config config/
    mi_cueva_4x4.yaml
95  python 05_utility_based_agent.py --config
    config/mi_cueva_4x4.yaml
96  ```
97
98  ## Criterios de aceptación
99
14
15  pi
16
17
18
19
20  gc
21
22  sc

Step 31  Score 969.0  CLIMBED
 4 | . . P .
 3 | . P . P
 2 | . . . .
 1 | . . . W
    1 2 3 4
Percept [None]
Reward +999 (climbed_with_gold)
-----
Result: climbed with gold  steps=31  score=969.0
PS C:\Users\marle\Git_IA\inteligencia-artificial\Agentes\project>
  
```

Figura 2: Ejecución del Agente de Reflejo Simple (02_simple_reflex_agent.py).

```

94 python 04_goal_based_agent.py --config config/
    mi_cueva_4x4.yaml
95 python 05_utility_based_agent.py --config
    config/mi_cueva_4x4.yaml
96
97
98 ## Criterios de aceptación

```

```

16 - [3,
17 - [3,
18 - [4,
19
20 gold: [2
21
22 scoring:
23 gold:

```

Problems Output Debug Console **Terminal** Ports

```

Step 31 Score 969.0 CLIMBED
4 | . . P .
3 | . P . P
2 | . . . .
1 | . . . W
   1 2 3 4
Percept [None]
Reward +999 (climbed_with_gold)
-----
Result: climbed with gold steps=31 score=969.0
PS C:\Users\marle\Git_IA\inteligencia-artificial\Agentes\project>

```

Figura 3: Ejecución del Agente Basado en Modelos (03_model_based_agent.py).

```

95 python 05_utility_based_agent.py --config
    config/mi_cueva_4x4.yaml
96
97
98 ## Criterios de aceptación

```

```

18 -
19
20 gold:
21
22 scoring:
23 gold:

```

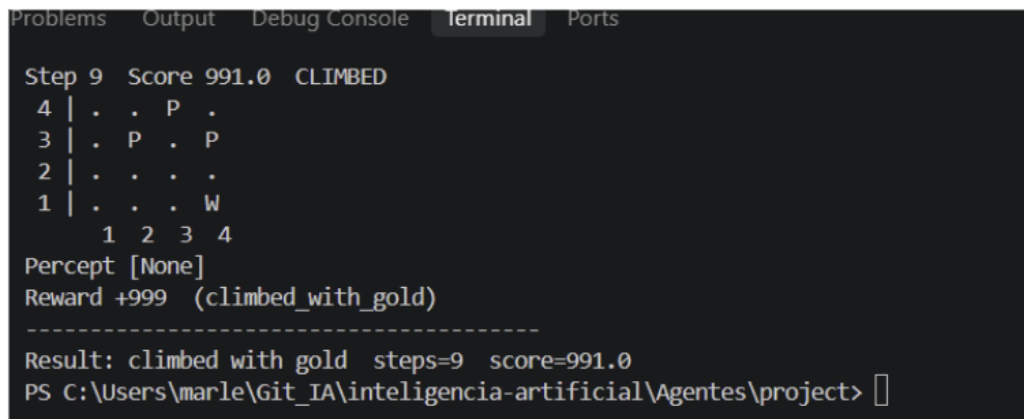
Problems Output Debug Console **Terminal** Ports

```

Step 17 Score 983.0 CLIMBED
4 | . . P .
3 | . P . P
2 | . . . .
1 | . . . W
   1 2 3 4
Percept [None]
Reward +999 (climbed_with_gold)
-----
Result: climbed with gold steps=17 score=983.0
PS C:\Users\marle\Git_IA\inteligencia-artificial\Agentes\project>

```

Figura 4: Ejecución del Agente Basado en Metas (04_goal_based_agent.py).



```
Problems  Output  Debug Console  terminal  Ports

Step 9  Score 991.0  CLIMBED
4 | . . P .
3 | . P . P
2 | . . . .
1 | . . . W
   1 2 3 4
Percept [None]
Reward +999 (climbed_with_gold)
-----
Result: climbed with gold  steps=9  score=991.0
PS C:\Users\marle\Git_IA\inteligencia-artificial\Agentes\project> 
```

Figura 5: Ejecución del Agente Basado en Utilidad (05.utility_based_agent.py).